



HTTP Headers, Dependencias y SSRF

Taller de Seguridad Web – OWASP Top 10

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

Agenda del día

🕒 Sesión 1 (2h)

Bloque	Tema
Repaso	Warm-up día 5
T13	HTTP Security Headers
Lab	Auditoría con securityheaders.com
T13-Fix	Helmet + CSP
Lab	Implementar Helmet

🕒 Sesión 2 (2h)

Bloque	Tema
T14	Dependencias vulnerables
Lab	npm audit + Snyk
T15	SSRF — Server-Side Request Forgery
Lab	SSRF conceptual + demo

Objetivos del día

Al terminar hoy serás capaz de:

1. 📋 Auditar las cabeceras de seguridad de cualquier web
2. 🛡️ Pasar de **puntuación F a A** en securityheaders.com con una línea de código
3. 📦 Detectar CVEs en tus dependencias con `npm audit`
4. 🌐 Entender cómo SSRF convierte al servidor en un proxy para el atacante

⚠️ **VulnMotors tiene puntuación F** en securityheaders.com. Al final del día la arreglaremos.

OWASP Top 10:2025 — Progreso

#	Categoría	Día(s)	Estado
A01	Broken Access Control	4	✓
A02	Security Misconfiguration (Headers)	6-7	🔥 HOY
A03	Supply Chain (Dependencies)	6, 8	🔥 HOY
A04	Cryptographic Failures	5	✓
A05	Injection	1-3	✓
A06	Insecure Design (SSRF)	4, 6	🔥 HOY ✓
A07	Authentication Failures	3-4	✓
A08	Software/Data Integrity Failures	8	
A09	Security Logging & Alerting	8	
A10	Mishandling of Exceptional Conditions	3	✓

6 categorías completadas · **Hoy: la superficie del servidor**

T13: HTTP Security Headers

Las 6 cabeceras que toda app necesita

¿Por qué importan las cabeceras?

Sin cabeceras de seguridad, el navegador **no sabe qué restricciones aplicar**:

Sin cabecera	Ataque posible
Sin <code>CSP</code>	XSS ejecuta JS sin restricción
Sin <code>HSTS</code>	SSL stripping (downgrade a HTTP)
Sin <code>X-Frame-Options</code>	Clickjacking (iframe invisible)
Sin <code>X-Content-Type-Options</code>	MIME sniffing (ejecutar archivos como JS)
Sin <code>Referrer-Policy</code>	URLs sensibles se filtran al siguiente sitio
Sin <code>Permissions-Policy</code>	La web accede a cámara/micrófono sin permiso
<code>CORS: *</code> (mal configurado)	Cualquier web puede llamar a tu API como el usuario

Las 6 cabeceras esenciales

1. Forzar HTTPS

Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

2. Bloquear XSS (inline scripts)

Content-Security-Policy: default-src 'self'; script-src 'self'

3. Prevenir clickjacking

X-Frame-Options: DENY

4. Evitar MIME sniffing

X-Content-Type-Options: nosniff

5. Controlar qué se envía como Referer

Referrer-Policy: strict-origin-when-cross-origin

6. Restringir APIs del navegador

Permissions-Policy: camera=(), microphone=(), geolocation=()

Content Security Policy (CSP) — La más poderosa

```
Content-Security-Policy:  
default-src 'self';  
script-src 'self';  
style-src 'self' 'unsafe-inline';  
img-src 'self' data: https;;  
connect-src 'self' https://api.example.com;
```

Lo que bloquea:

- `<script>alert(1)</script>` → ❌ Bloqueado (inline script)
- `<img src=x onerror="...">` → ❌ Bloqueado (inline event handler)
- `<script src="https://evil.com/malware.js">` → ❌ Bloqueado (origen no permitido)

✅ **CSP es la última línea de defensa contra XSS** — incluso si inyectas HTML, el JS no se ejecuta.

Clickjacking — El ataque que X-Frame-Options previene

```
<!-- El atacante carga tu web en un iframe invisible: -->
<style>
  iframe { opacity: 0; position: absolute; top: 0; left: 0;
           width: 100%; height: 100%; z-index: 2; }
  .bait { z-index: 1; position: relative; }
</style>

<div class="bait"><h1>¡Clic aquí para ganar!</h1></div>
<iframe src="https://bank.com/delete-account"></iframe>
```

El usuario cree que hace clic en "ganar" pero realmente hace clic en "borrar cuenta".

Fix: `X-Frame-Options: DENY` o `Content-Security-Policy: frame-ancestors 'none'`

El fix rápido — Helmet.js (una línea)

```
import helmet from 'helmet';

// Una línea añade las 6 cabeceras:
app.use(helmet());

// Personalizar CSP:
app.use(helmet.contentSecurityPolicy({
  directives: {
    defaultSrc: ["'self'"],
    scriptSrc: ["'self'"],
    styleSrc: ["'self'", "'unsafe-inline'"],
    imgSrc: ["'self'", "data:", "https:"],
  }
}));
```

Resultado en securityheaders.com: F → A 🎉

CORS — Cross-Origin Resource Sharing

Same-Origin Policy: por defecto, el navegador **bloquea** peticiones JS a otro dominio.

```
Tu frontend: https://miapp.com
Tu API:      https://api.miapp.com   ← ¡Origen diferente! Navegador bloquea.
```

CORS = cabeceras que permiten excepciones controladas.

Cabecera	Qué permite
<code>Access-Control-Allow-Origin</code>	Qué orígenes pueden llamar a tu API
<code>Access-Control-Allow-Methods</code>	Qué métodos HTTP acepta (GET, POST...)
<code>Access-Control-Allow-Headers</code>	Qué cabeceras custom acepta
<code>Access-Control-Allow-Credentials</code>	Si envía cookies cross-origin

CORS — Configuración segura vs insegura

✗ Inseguro — todo abierto:

```
// Cualquier web del mundo puede
// llamar a tu API:
app.use(cors({ origin: '*' }));

// PEOR AÚN: con credenciales
app.use(cors({
  origin: '*',
  credentials: true // ← PELIGRO
}));
```

⚠ `origin: '*'` + `credentials: true` = cualquier web puede actuar como el usuario.

✓ Seguro — allowlist explícita:

```
import cors from 'cors';

const ALLOWED = [
  'https://miapp.com',
  'https://admin.miapp.com'
];

app.use(cors({
  origin: ALLOWED,
  methods: ['GET', 'POST', 'PUT'],
  credentials: true
}));
```

Solo orígenes conocidos pueden hacer peticiones.

CORS — El error más común

El navegador muestra en la consola:

```
✗ Access to fetch at 'https://api.miapp.com/users'  
  from origin 'https://otro.com' has been blocked by CORS policy:  
  No 'Access-Control-Allow-Origin' header is present.
```

¿Qué significa? Tu API no tiene cabeceras CORS → el navegador protege al usuario bloqueando la respuesta.

¿Solución correcta? Configurar tu API para que solo permita orígenes legítimos.

¿Solución INCORRECTA? Poner `Access-Control-Allow-Origin: *` "para que funcione" → estás abriendo tu API al mundo.

💡 **CORS NO es un firewall.** Es una política del **navegador**. Un atacante con curl/Postman ignora CORS completamente. Pero SÍ protege contra ataques desde webs maliciosas que usen el navegador del usuario.

Lab T13 — Auditar y arreglar headers

Parte 1 — Auditoría:

1. Ir a securityheaders.com
2. Analizar `https://vulnapp-owasp-01.azurewebsites.net`
3. ¿Qué nota obtiene? ¿Qué cabeceras faltan?

Parte 2 — Verificar con curl:

```
curl -I https://vulnapp-owasp-01.azurewebsites.net  
# Buscar: Strict-Transport-Security, Content-Security-Policy, X-Frame-Options
```

Parte 3 — El fix (código):

```
npm install helmet
```

```
import helmet from 'helmet';  
app.use(helmet());
```

 **Objetivo:** Pasar de F a A en securityheaders.com con una línea de código.

T14: Dependencias Vulnerables

Tu código es el 3% — el otro 97% está en node_modules

El problema — Supply chain

Tu aplicación:

Tu código: ~5.000 líneas
node_modules: ~500.000 líneas

97% del código NO lo has
escrito tú

Ataques a la cadena de suministro:

- **Typosquatting:** `loadash` (fake) vs `lodash` (real)
- **Dependency confusion:** package privado con nombre público
- **Maintainer hijack:** `event-stream` 2018, `colors.js` 2022
- **CVEs conocidos:** Log4Shell (CVSS 10.0)

💀 **Log4Shell (2021):** Una vulnerabilidad en una dependencia afectó al 93% de entornos cloud empresariales.

npm audit — Tu primera línea de defensa

```
# Escanear vulnerabilidades conocidas:
npm audit

# Resultado típico:
# 3 vulnerabilities (1 moderate, 1 high, 1 critical)
#
# jsonwebtoken <=8.5.1 | High | JWT signature bypass
# express <4.19.2 | Moderate | Open redirect

# Arreglar automáticamente lo que se pueda:
npm audit fix

# Ver detalles:
npm audit --json | jq '.vulnerabilities | keys'
```

`npm ci` vs `npm install` — Reproducibilidad

Comando	Modifica lockfile	Uso
<code>npm install</code>	✅ Sí (puede actualizar)	Desarrollo local
<code>npm ci</code>	❌ No (falla si no coincide)	CI/CD y producción

```
# En CI/CD SIEMPRE usar npm ci:  
npm ci # instala EXACTO lo que dice package-lock.json
```

⚠️ `npm install` puede introducir versiones nuevas silenciosamente. En producción → **solo** `npm ci`.

Dependabot – Actualizaciones automáticas

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/VulnApp"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 10
```

SBOM (Software Bill of Materials):

```
# Generar inventario completo de dependencias:
npx @cyclonedx/cyclonedx-npm --output-file sbom.json
```



Lab T14 — Auditar dependencias

```
cd VulnApp
```

```
# 1. Ejecutar auditoría
```

```
npm audit
```

```
# 2. ¿Cuántas vulnerabilidades hay? ¿Qué severidad?
```

```
# 3. Intentar fix automático
```

```
npm audit fix
```

```
# 4. ¿Quedaron vulnerabilidades sin resolver?
```

```
npm audit
```



Documenta: nombre del paquete vulnerable, CVE, severidad, si se pudo arreglar automáticamente.

T15: SSRF — Server-Side Request Forgery

Cuando el servidor hace peticiones por ti

¿Qué es SSRF?

El servidor hace **peticiones HTTP** a **URLs controladas por el atacante**:

```
// src/routes/fetch.ts - VULNERABLE
app.get('/fetch-image', async (req, res) => {
  const url = req.query.url as string;
  const response = await fetch(url); // ← ¡El usuario controla la URL!
  const data = await response.text();
  res.json({ data });
});
```

El atacante envía:

```
GET /fetch-image?url=http://localhost:3000/debug
```

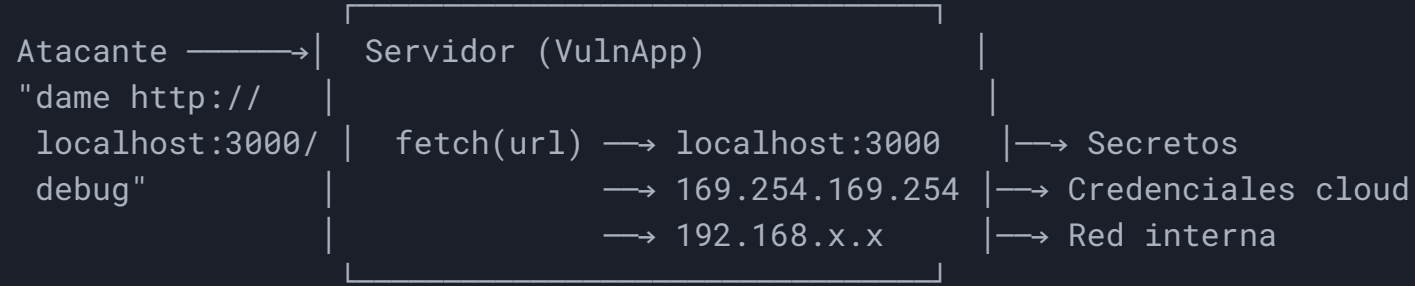
→ El servidor accede a su **propio endpoint** `/debug` y devuelve secretos al atacante.

Objetivos típicos de SSRF

Objetivo	URL	Qué obtiene
AWS IMDS	<code>http://169.254.169.254/latest/meta-data/</code>	Credenciales IAM temporales
GCP Metadata	<code>http://metadata.google.internal/</code>	Tokens de servicio
Servicios internos	<code>http://localhost:6379</code>	Redis/BD sin auth
Red interna	<code>http://192.168.1.1/admin</code>	Paneles de administración
Archivos locales	<code>file:///etc/passwd</code>	Archivos del sistema

💀 **Capital One 2019:** SSRF → credenciales AWS → 100 millones de registros de clientes.

SSRF – El patrón de ataque



¿Por qué funciona?

1. El servidor tiene acceso a la **red interna** (localhost, metadata, otros servicios)
2. El atacante NO tiene ese acceso (firewall, VPN, security groups)
3. El servidor **ejecuta la petición en nombre del atacante** → actúa como proxy
4. Los servicios internos confían en peticiones desde localhost → **sin autenticación**

💡 SSRF es especialmente grave en cloud: el endpoint de metadatos (`169.254.169.254`) devuelve credenciales temporales que dan acceso a S3, RDS, etc.

SSRF en VulnApp — Demostración en vivo

```
URL="https://vulnapp-owasp-01.azurewebsites.net/fetch-image"

# 1. Acceder a endpoint interno (secretos del servidor):
curl -s "$URL?url=http://localhost:3000/debug"
# → Devuelve variables de entorno y secretos

# 2. Acceder a info de la API interna:
curl -s "$URL?url=http://localhost:3000/api"
# → Lista todas las rutas de la app

# 3. Intentar metadatos AWS/Azure:
curl -s "$URL?url=http://169.254.169.254/latest/meta-data/"
curl -s "$URL?url=http://169.254.169.254/metadata/instance?api-version=2021-02-01"

# 4. Port scanning interno:
curl -s "$URL?url=http://127.0.0.1:22"      # SSH
curl -s "$URL?url=http://127.0.0.1:6379"    # Redis

# 5. Protocolo file:///:
curl -s "$URL?url=file:///etc/passwd"
```

💀 **El servidor se convierte en un proxy para el atacante.** Accede a recursos internos que normalmente están protegidos por firewall.

El fix – Allowlist + validación de IP

```
// src/routes/fetch.ts – SEGURO
const ALLOWED_HOSTS = new Set([
  'images.unsplash.com', 'via.placeholder.com',
  'picsum.photos', 'i.imgur.com'
]);

app.get('/fetch-image', async (req, res) => {
  const url = req.query.url as string;
  const parsed = new URL(url);

  // 1. Solo protocolos HTTP/HTTPS
  if (!['http:', 'https:'].includes(parsed.protocol)) {
    return res.status(403).json({ error: 'Protocolo no permitido' });
  }

  // 2. Solo hosts permitidos
  if (!ALLOWED_HOSTS.has(parsed.hostname)) {
    return res.status(403).json({ error: 'Host no permitido' });
  }

  const response = await fetch(url);
  res.json({ data: await response.text() });
});
```

Lab T15 — SSRF en vivo

```
URL="https://vulnapp-owasp-01.azurewebsites.net/fetch-image"
```

```
# 1. Acceso a endpoint interno - secretos del servidor:
```

```
curl -s "$URL?url=http://localhost:3000/debug"
```

```
# 2. Enumerar la API interna:
```

```
curl -s "$URL?url=http://localhost:3000/api"
```

```
# 3. Intentar metadatos cloud:
```

```
curl -s "$URL?url=http://169.254.169.254/latest/meta-data/"
```

```
# 4. Protocolo file:/// (acceso a archivos locales):
```

```
curl -s "$URL?url=file:///etc/passwd"
```

Después del fix (**VULN_SSRF=0**):

```
# Todos devolverán 403 - solo dominios de imágenes permitidos:
```

```
curl -s "$URL?url=http://localhost:3000/debug" # → 403 Host no permitido
```

```
curl -s "$URL?url=https://picsum.photos/200" # → 200 OK (allowlisted)
```

 **Objetivo:** Demuestra que el servidor actúa como proxy. Exfiltra secretos vía SSRF.

 **Attack Lab:** `/attacker.html` → pestaña SSRF con payloads precargados.

Labs del día — resumen

Lab	Qué harás	Herramienta
T13	Auditar VulnMotors en securityheaders.com → nota F	Navegador
T13-Fix	Instalar helmet → re-auditar → nota A	npm + editor
T14	Ejecutar <code>npm audit</code> y analizar vulnerabilidades	Terminal
T15	SSRF en vivo: <code>GET /fetch-image?url=http://localhost:3000/debug</code>	curl + Attack Lab

 **URL:** `https://vulnapp-owasp-01.azurewebsites.net`

 **Attack Lab:** `https://vulnapp-owasp-01.azurewebsites.net/attacker.html`

 **Herramientas:** securityheaders.com · [Snyk](https:// snyk.com) · curl

Resumen del día

Tema	Problema	Solución
Headers	Sin CSP/HSTS → XSS libre, SSL stripping	<code>helmet()</code> → 6 headers en una línea
Dependencias	97% código ajeno con CVEs	<code>npm audit</code> + Dependabot + SBOM
SSRF	Servidor como proxy del atacante	Allowlist de hosts + bloquear protocolos no-HTTP

🏆 **Logro del día:** Sabéis auditar cabeceras, detectar dependencias vulnerables y entender cómo SSRF da acceso a redes internas.